

我们首先来看第一章节

从这一章开始，我们开始迈入单片机的世界。在我们开始这一章具体的学习之前，有必要给大家先说明一下。在以后的系列文章中，我们将以 51 内核的单片机为载体，C 语言为编程语言，开发环境为 KEIL uv3。至于为什么选用 C 语言开发，好处不言而喻，开发速度快，效率高，代码可复用率高，结构清晰，尤其是在大型的程序中，而且随着编译器的不断升级，其编译后的代码大小与汇编语言的差距越来越小。而关于 C 语言和汇编之争，就像那个啥，每隔一段时间总会有人挑起这个话题，如果你感兴趣，可以到网上搜索相关的帖子自行阅读。不是说汇编不重要，在很多对时序要求非常高的场合，需要利用汇编语言和 C 语言混合编程才能够满足系统的需求。在我们学习掌握 C 语言的同时，也还需要利用闲余的时间去学习了解汇编语言。

1.从点亮 LED(发光二极管)开始

在市面上众多的单片机学习资料中，最基础的实验无疑于点亮 LED 了，即控制单片机的 I/O 的电平的变化。

如同如下实例代码一般

```
void main(void)
{
    LedInit();
    While(1)
    {
        LED = ON;
        DelayMs(500);
        LED = OFF;
        DelayMs(500);
    }
}
```

程序很简单，从它的结构可以看出，LED 先点亮 500MS，然后熄灭 500MS，如此循环下去，形成的效果就是 LED 以 1HZ 的频率进行闪烁。下面让我们分析上面的程序有没有什么问题。

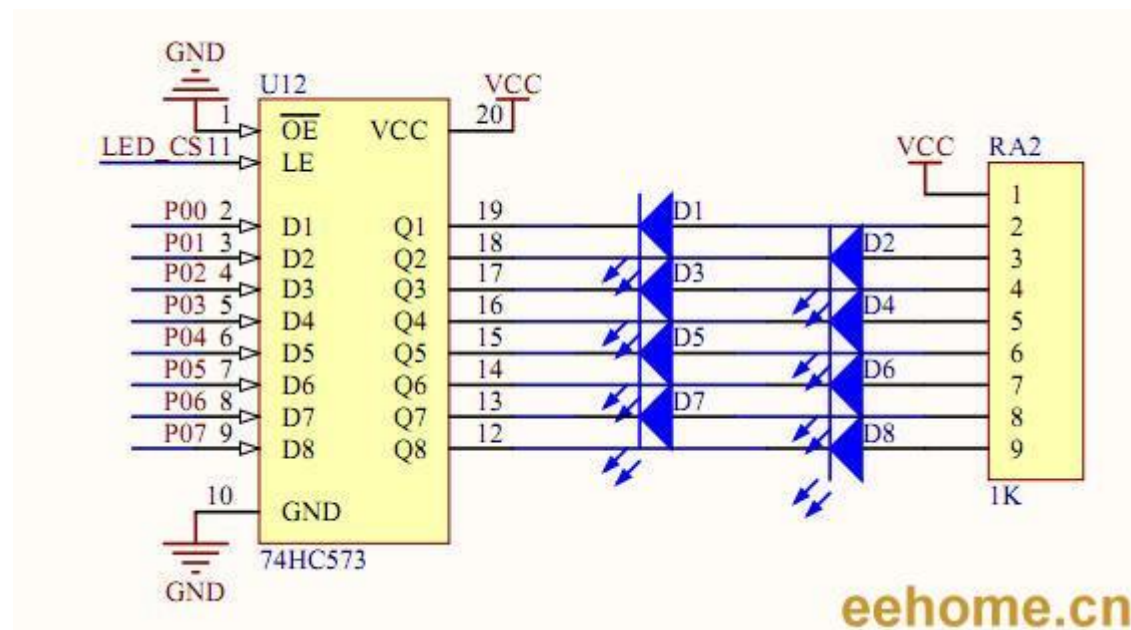
看来看出，好像很正常的啊，能有什么问题呢？这个时候我们应该换一个思路去想了。试想，整个程序除了控制 LED = ON ； LED = OFF ； 这两条语句外，其余的时间，全消耗在了 DelayMs(500)这两个函数上。而在实际应用系统中是没有哪个系统只闪烁一只 LED 就其它什么事情都不做了的。因此，在这里我们要想办法，把 CPU 解放出来，让它不要白白浪费 500MS 的延时等待时间。宁可让它一遍又一遍的扫描看有哪些任务需要执行，也不要让它停留在某个地方空转消耗 CPU 时间。

从上面我们可以总结出

- (1) 无论什么时候我们都要以实际应用的角度去考虑程序的编写。
- (2) 无论什么时候都不要让 CPU 白白浪费等待，尤其是延时(超过 1MS)这样的地方。

下面让我们从另外一个角度来考虑如何点亮一颗 LED。
先看看我们的硬件结构是什么样子的。

我手上的单片机板子是电子工程师之家的开发的学习板。就以它的实际硬件连接图来分析吧。如下图所示



(原文件名:led.jpg)

一般的 LED 的正常发光电流为 10~20mA 而低电流 LED 的工作电流在 2mA 以下（亮度与普通发光管相同）。在上图中我们可知，当 Q1~Q8 引脚上面的电平为低电平时，LED 发光。通过 LED 的电流约为 $(VCC - V_d) / RA2$ 。其中 V_d 为 LED 导通后的压降，约为 1.7V 左右。这个导通压降根据 LED 颜色的不同，以及工作电流的大小的不同，会有一定的差别。下面一些参数是网上有人测出来的，供大家参考。

红色的压降为 1.82-1.88V，电流 5-8mA，

绿色的压降为 1.75-1.82V，电流 3-5mA，

橙色的压降为 1.7-1.8V，电流 3-5mA

兰色的压降为 3.1-3.3V，电流 8-10mA，

白色的压降为 3-3.2V，电流 10-15mA，

(供电电压 5V，LED 直径为 5mm)

74HC573 真值表如下：

FUNCTION TABLE

Inputs			Output
Output Enable	Latch Enable	D	Q
L	H	H	H
L	H	L	L
L	L	X	no change
H	X	X	Z

X = don't care

Z = high impedance

eehome.cn

(原文件名:74hc573.jpg)

通过这个真值表我们可以看出。当 OutputEnable 引脚接低电平的时候, 并且 LatchEnable 引脚为高电平的时候, Q 端电平与 D 端电平相同。结合我们的 LED 硬件连接图可以知道 LED_CS 端为高电平时, P0 口电平的变化即 Q 端的电平的变化, 进而引起 LED 的亮灭变化。由于单片机的驱动能力有限, 在此, 74HC573 的主要作用就是起一个输出驱动的作用。需要注意的是, 通过 74HC573 的最大电流是有限制的, 否则可能会烧坏 74HC573 这个芯片。

上面这个图是从 74HC573 的 DATASHEET 中截取出来的, 从上可以看出, 每个引脚允许通过的最大电流为 35mA 整个芯片允许通过的最大电流为 75mA。在我们设计相应的驱动电路时候, 这些参数是相当重要的, 而且是最容易被初学者所忽略的地方。同时在设计的时候, 要留出一定量的余量出来, 不能说单个引脚允许通过的电流为 35mA, 你就设计为 35mA, 这个时候你应该把设计的上限值定在 20mA 左右才能保证能够稳定的工作。

(设计相应驱动电路时候, 应该仔细阅读芯片的数据手册, 了解每个引脚的驱动能力, 以及整个芯片的驱动能力)

了解了相应的硬件后, 我们再来编写驱动程序。

首先定义 LED 的接口

```
#define LED_P0
```

然后为亮灭常数定义一个宏, 由硬件连接图可以, 当 P0 输出为低电平时 LED 亮, P0 输出为高电平时, LED 熄灭。

```
#define LED_ON() LED = 0x00 //所有 LED 亮
```

```
#define LED_OFF() LED = 0xff //所有 LED 熄灭
```

下面到了重点了, 究竟该如何释放 CPU, 避免其做延时空等待这样的事情呢。很简单, 我们为系统产生一个 1MS 的时标。假定 LED 需要亮 500MS, 熄灭 500MS, 那么我们可以对这个 1MS 的时标进行计数, 当这个计数值达到 500 时候, 清零该计数值, 同时把 LED 的状态改变。

```
unsigned int g_u16LedTimeCount = 0; //LED 计数器
```

```
unsigned char g_u8LedState = 0; //LED 状态标志, 0 表示亮, 1 表示熄灭
```

```

void LedProcess(void)
{
    if(0 == g_u8LedState) //如果 LED 的状态为亮，则点亮 LED
    {
        LED_ON();
    }
    else                //否则熄灭 LED
    {
        LED_OFF();
    }
}

```

```

void LedStateChange(void)
{
    if(g_bSystemTime1Ms)        //系统 1MS 时标到
    {
        g_bSystemTime1Ms = 0;
        g_u16LedTimeCount++;    //LED 计数器加一
        if(g_u16LedTimeCount >= 500) //计数达到 500,即 500MS 到了,改变 LED 的状态。
        {
            g_u16LedTimeCount = 0;
            g_u8LedState = !g_u8LedState;
        }
    }
}

```

上面有一个变量没有提到，就是 g_bSystemTime1Ms。这个变量可以定义为位变量或者是其它变量，在我们的定时器中断函数中对其置位，其它函数使用该变量后，应该对其复位(清 0)。

我们的主函数就可以写成如下形式(示意代码)

```

void main(void)
{
    while(1)
    {
        LedProcess();
        LedStateChange();
    }
}

```

因为 LED 的亮或者灭依赖于 LED 状态变量(g_u8LedState)的改变，而状态变量的改变，又依赖于 LED 计数器的计数值(g_u16LedTimeCount，只有计数值达到一定后，状态变量才改变)所以，两个函数都没有堵塞 CPU 的地方。让我们来从头到尾分析一遍整个程序的流程。

程序首先执行 LedProcess() ;函数

因为 g_u8LedState 的初始值为 0 (见定义, 对于全局变量, 在定义的时候最好给其一个确定的值)所以 LED 被点亮, 然后退出 LedStateChange()函数, 执行下一个函数 LedStateChange()在函数 LedStateChange()内部首先判断 1MS 的系统时标是否到了, 如果没有到就直接退出函数, 如果到了, 就把时标清 0 以便下一个时标消息的到来, 同时对 LED 计数器加一, 然后再判断 LED 计数器是否到达我们预先想要的值 500, 如果没有, 则退出函数, 如果有, 对计数器清 0, 以便下次重新计数, 同时把 LED 状态变量取反, 然后退出函数。

由上面整个流程可以知道, CPU 所做的事情, 就是对一些计数器加一, 然后根据条件改变状态, 再根据这个状态来决定是否点亮 LED。这些函数执行所花的时间都是相当短的, 如果主程序中还有其它函数, 则 CPU 会顺次往下执行下去。对于其它的函数(如果有的话)也要采取同样的措施, 保证其不堵塞 CPU, 如果全部基于这种方法设计, 那么对于不是非常庞大的系统, 我们的系统依旧可以保证多个任务(多个函数)同时执行。系统的实时性得到了一定的保证, 从宏观上看来, 就是多个任务并发执行。

好了, 这一章就到此为止, 让我们总结一下, 究竟有哪些需要注意的吧。

- (1) 无论什么时候我们都要以实际应用的角度去考虑程序的编写。
- (2) 无论什么时候都不要让 CPU 白白浪费等待, 尤其是延时(超过 1MS)这样的地方。
- (3) 设计相应驱动电路时候, 应该仔细阅读芯片的数据手册, 了解每个引脚的驱动能力, 以及整个芯片的驱动能力
- (4) 最重要的是, 如何去释放 CPU(参考本章的例子), 这是写出合格程序的基础。

附完整程序代码(基于电子工程师之家的单片机开发板)

```
#include<reg52.h>

sbit LED_SEG = P1^4; //数码管段选
sbit LED_DIG = P1^5; //数码管位选
sbit LED_CS11 = P1^6; //led 控制位
sbit ir=P1^7;

#define LED P0 //定义 LED 接口
bit g_bSystemTime1Ms = 0; // 1MS 系统时标
unsigned int g_u16LedTimeCount = 0; //LED 计数器
unsigned char g_u8LedState = 0; //LED 状态标志, 0 表示亮, 1 表示熄灭

#define LED_ON() LED = 0x00; //所有 LED 亮
#define LED_OFF() LED = 0xff; //所有 LED 熄灭

void Timer0Init(void)
{
    TMOD &= 0xf0;
    TMOD |= 0x01; //定时器 0 工作方式 1
```

```

    TH0 = 0xfc;    //定时器初始值
    TL0 = 0x66;
    TR0 = 1;
    ET0 = 1;
}
void LedProcess(void)
{
    if(0 == g_u8LedState) //如果 LED 的状态为亮, 则点亮 LED
    {
        LED_ON();
    }
    else                //否则熄灭 LED
    {
        LED_OFF();
    }
}

void LedStateChange(void)
{
    if(g_bSystemTime1Ms)    //系统 1MS 时标到
    {
        g_bSystemTime1Ms = 0;
        g_u16LedTimeCount++;    //LED 计数器加一
        if(g_u16LedTimeCount >= 500) //计数达到 500,即 500MS 到了,改变 LED 的状态。
        {
            g_u16LedTimeCount = 0;
            g_u8LedState = !g_u8LedState ;
        }
    }
}

void main(void)
{
    Timer0Init();
    EA = 1;
    LED_CS11 = 1; //74HC595 输出允许
    LED_SEG = 0; //数码管段选和位选禁止(因为它们和 LED 共用 P0 口)
    LED_DIG = 0;
    while(1)
    {
        LedProcess();
        LedStateChange();
    }
}

```

```

void Time0Isr(void) interrupt 1
{
    TH0 = 0xfc;          //定时器重新赋初值
    TL0 = 0x66;
    g_bSystemTime1Ms = 1; //1MS 时标标志位置位
}

```

“从单片机初学者迈向单片机工程师”

第三章----模块化编程初识

好的开始是成功的一半

通过上一章的学习,我想你已经掌握了如何在程序中释放 CPU 了。希望能够继续坚持下去。一个良好的开始是成功的一半。我们今天所做的一切都是为了在单片机编程上做的更好。在谈论今天的主题之前,先说下我以前的一些经历。在刚开始接触到 C 语言程序的时候,由于学习内容所限,写的程序都不是很大,一般也就几百行而矣。所以所有的程序都完成在一个源文件里面。记得那时候大一参加学校里的一个电子设计大赛,调试了一个多星期,所有程序加起来大概将近 1000 行,长长的一个文件,从上浏览下来都要好半天。出了错误简单的语法错误还好定位,其它一些错误,往往找半天才找的到。那个时候开始知道了模块化编程这个东西,也尝试着开始把程序分模块编写。最开始是把相同功能的一些函数(譬如 1602 液晶的驱动)全部写在一个头文件(.h)文件里面,然后需要调用的地方包含进去,但是很快发现这种方法有其局限性,很容易犯重复包含的错误。

而且调用起来也很不方便。很快暑假的电子设计大赛来临了,学校对我们的单片机软件编程进行了一些培训。由于学校历年来参加国赛和省赛,因此积累了一定数量的驱动模块,那些日子,老师每天都会布置一定量的任务,让我们用这些模块组合起来,完成一定功能。而正是那些日子模块化编程的培训,使我对于模块化编程有了更进一步的认识。并且程序规范也开始慢慢注意起来。此后的日子,无论程序的大小,均采用模块化编程的方式去编写。很长一段时间以来,一直有单片机爱好者在 QQ 上和我一起交流。有时候,他们会发过来一些有问题的程序源文件,让我帮忙修改一下。同样是长长的一个文件,而且命名极不规范,从头看起来,着实是痛苦,说实话,还真不如我重新给他们写一个更快一些,此话到不假,因为手头积累了一定量的模块,在完成一个新的系统时候,只需要根据上层功能需求,在底层模块的支持下,可以很快方便的完成。而不需要从头到尾再一砖一瓦的重新编写。藉此,也可以看出模块化编程的一个好处,就是可重复利用率高。下面让我们揭开模块化神秘面纱,一窥其真面目。

C 语言源文件 *.c

提到 C 语言源文件,大家都不会陌生。因为我们平常写的程序代码几乎都在这个 XX.C 文件里面。编译器也是以此文件来进行编译并生成相应的目标文件。作为模块化编程的组成基础,我们所要实现的所有功能的源代码均在这个文件里。理想的模块化应该可以看成是一个黑盒子。即我们只关心模块提供的功能,而不管模块内部的实现细节。好比我们买了一部手机,我们只需要会用手机提供的功能即可,不需要知晓它是如何把短信发出去的,如何响应我们按键的输入,这些过程对我们用户而言,就是是一个黑盒子。

在大规模程序开发中，一个程序由很多个模块组成，很可能，这些模块的编写任务被分配到不同的人。而你在编写这个模块的时候很可能就需要利用到别人写好的模块的借口，这个时候我们关心的是，它的模块实现了什么样的接口，我该如何去调用，至于模块内部是如何组织的，对于我而言，无需过多关注。而追求接口的单一性，把不需要的细节尽可能对外部屏蔽起来，正是我们所需要的地方。

C 语言头文件 *.h

谈及到模块化编程，必然会涉及到多文件编译，也就是工程编译。在这样的一个系统中，往往会有多个 C 文件，而且每个 C 文件的作用不尽相同。在我们的 C 文件中，由于需要对外提供接口，因此必须有一些函数或者是变量提供给外部其它文件进行调用。

假设我们有一个 LCD.C 文件，其提供最基本的 LCD 的驱动函数

```
LcdPutChar(char cNewValue); //在当前位置输出一个字符
```

而在我们的另外一个文件中需要调用此函数，那么我们该如何做呢？

头文件的作用正是在此。可以称其为一份接口描述文件。其文件内部不应该包含任何实质性的函数代码。我们可以把这个头文件理解成为一份说明书，说明的内容就是我们的模块对外提供的接口函数或者是接口变量。同时该文件也包含了一些很重要的宏定义以及一些结构体的信息，离开了这些信息，很可能就无法正常使用接口函数或者是接口变量。但是总的原则是：不该让外界知道的信息就不应该出现在头文件里，而外界调用模块内接口函数或者是接口变量所必须的信息就一定要出现在头文件里，否则，外界就无法正确的调用我们提供的接口功能。因而为了让外部函数或者文件调用我们提供的接口功能，就必须包含我们提供的这个接口描述文件----即头文件。同时，我们自身模块也需要包含这份模块头文件(因为其包含了模块源文件中所需要的宏定义或者是结构体)，好比我们平常所用的文件都是一式三份一样，模块本身也需要包含这个头文件。

下面我们来定义这个头文件，一般来说，头文件的名字应该与源文件的名字保持一致，这样我们便可以清晰的知道哪个头文件是哪个源文件的描述。

于是便得到了 LCD.C 的头文件 LCD.h 其内容如下。

```
#ifndef _LCD_H_
#define _LCD_H_
extern LcdPutChar(char cNewValue);
#endif
```

这与我们在源文件中定义函数时有点类似。不同的是，在其前面添加了 extern 修饰符表明其是一个外部函数，可以被外部其它模块进行调用。

```
#ifndef _LCD_H_
#define _LCD_H_
#endif
```

这个几条条件编译和宏定义是为了防止重复包含。假如有两个不同源文件需要调用 LcdPutChar(char cNewValue)这个函数，他们分别都通过#include "Lcd.h"把这个头文件包含了进去。在第一个源文件进行编译时候，由于没有定义过 _LCD_H_ 因此 #ifndef _LCD_H_ 条件成立，于是定义_LCD_H_ 并将下面的声明包含进去。在第二个文件编译时候，由于第一个文件包含时候，已经将_LCD_H_定义过了。因此#ifndef _LCD_H_ 不成立，整个头文件内容就没有被包含。假设没有这样的条件编译语句，那么两个文件都包含了 extern LcdPutChar(char cNewValue); 就会引起重复包含的错误。

不得不说的 typedef

很多朋友似乎习惯了程序中利用如下语句来对数据类型进行定义

```
#define uint unsigned int
#define uchar unsigned char
```

然后在定义变量的时候 直接这样使用

```
uint g_nTimeCounter = 0;
```

不可否认，这样确实很方便，而且对于移植起来也有一定的方便性。但是考虑下面这种情况你还会这么认为吗？

```
#define PINT unsigned int * //定义 unsigned int 指针类型
PINT g_npTimeCounter, g_npTimeState;
```

那么你到底是定义了两个 unsigned int 型的指针变量，还是一个指针变量，一个整形变量呢？而你的初衷又是什么呢，想定义两个 unsigned int 型的指针变量吗？如果是这样，那么估计过不久就会到处抓狂找错误了。

庆幸的是 C 语言已经为我们考虑到了这一点。typedef 正是为此而生。为了给变量起一个别名我们可以用如下的语句

```
typedef unsigned int uint16; //给指向无符号整形变量起一个别名 uint16
typedef unsigned int * puint16; //给指向无符号整形变量指针起一个别名 puint16
在我们定义变量时候便可以这样定义了：
```

```
uint16 g_nTimeCounter = 0; //定义一个无符号的整形变量
puint16 g_npTimeCounter; //定义一个无符号的整形变量的指针
```

在我们使用 51 单片机的 C 语言编程的时候，整形变量的范围是 16 位，而在基于 32 的微处理下的整形变量是 32 位。倘若我们在 8 位单片机下编写的一些代码想要移植到 32 位的处理器上，那么很可能我们就需要在源文件中到处修改变量的类型定义。这是一件庞大的工作，为了考虑程序的可移植性，在一开始，我们就应该养成良好的习惯，用变量的别名进行定义。

如在 8 位单片机的平台下，有如下一个变量定义

```
uint16 g_nTimeCounter = 0;
    如果移植 32 单片机的平台下，想要其的范围依旧为 16 位。
    可以直接修改 uint16 的定义，即
typedef unsigned short int uint16;
    这样就可以了，而不需要到源文件处处寻找并修改。
```

将常用的数据类型全部采用此种方法定义，形成一个头文件，便于我们以后编程直接调用。

文件名 MacroAndConst.h

其内容如下：

```
#ifndef _MACRO_AND_CONST_H_
#define _MACRO_AND_CONST_H_

typedef unsigned int uint16;
typedef unsigned int UINT;
typedef unsigned int uint;
typedef unsigned int UINT16;
typedef unsigned int WORD;
typedef unsigned int word;
```

```

typedef    int        int16;
typedef    int        INT16;
typedef    unsigned long  uint32;

typedef    unsigned long  UINT32;
typedef    unsigned long  DWORD;
typedef    unsigned long  dword;
typedef    long           int32;
typedef    long           INT32;
typedef    signed char    int8;
typedef    signed char    INT8;
typedef    unsigned char  byte;
typedef    unsigned char  BYTE;
typedef    unsigned char  uchar;
typedef    unsigned char  UINT8;
typedef    unsigned char  uint8;
typedef    unsigned char  BOOL;

#endif

```

至此，似乎我们对于源文件和头文件的分工以及模块化编程有那么一点概念了。那么让我们趁热打铁，将上一章的我们编写的 LED 闪烁函数进行模块划分并重新组织进行编译。

在上一章中我们主要完成的功能是 P0 口所驱动的 LED 以 1Hz 的频率闪烁。其中用到了定时器，以及 LED 驱动模块。因而我们可以简单的将整个工程分成三个模块，定时器模块，LED 模块，以及主函数
对应的文件关系如下

```

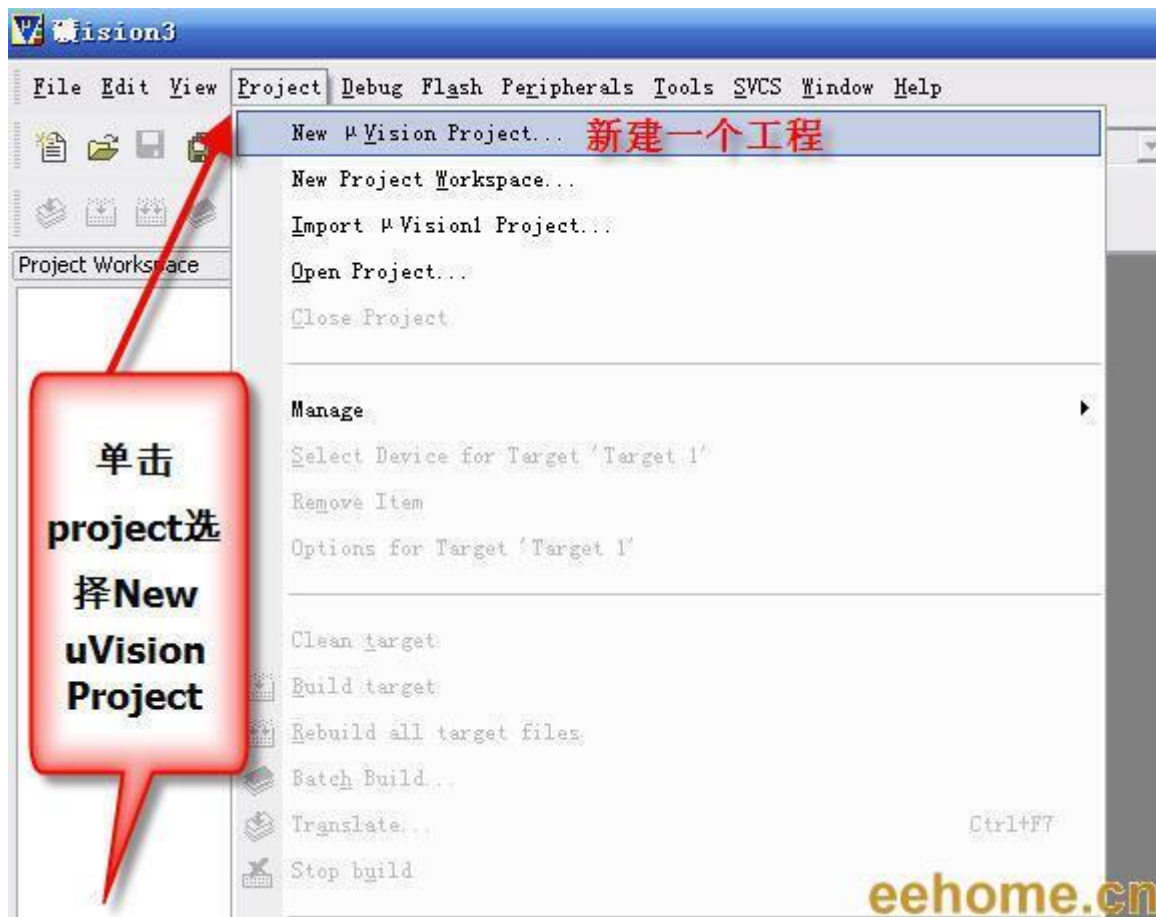
main.c
Timer.c  --&#61664;Timer.h
Led.c    --&#61664;Led.h

```

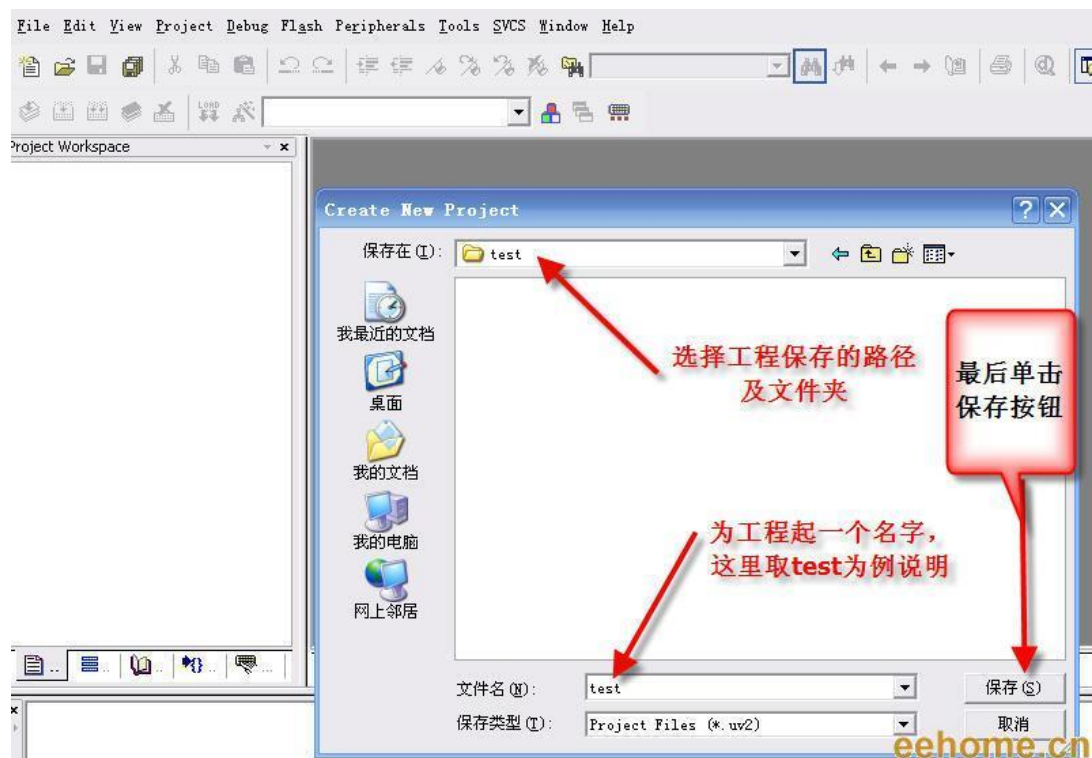
在开始重新编写我们的程序之前，先给大家讲一下如何在 KEIL 中建立工程模板吧，这个模板是我一直沿用至今。希望能够给大家一点启发。

下面的内容就主要以图片为主了。同时辅以少量文字说明。

我们以芯片 AT89S52 为例。



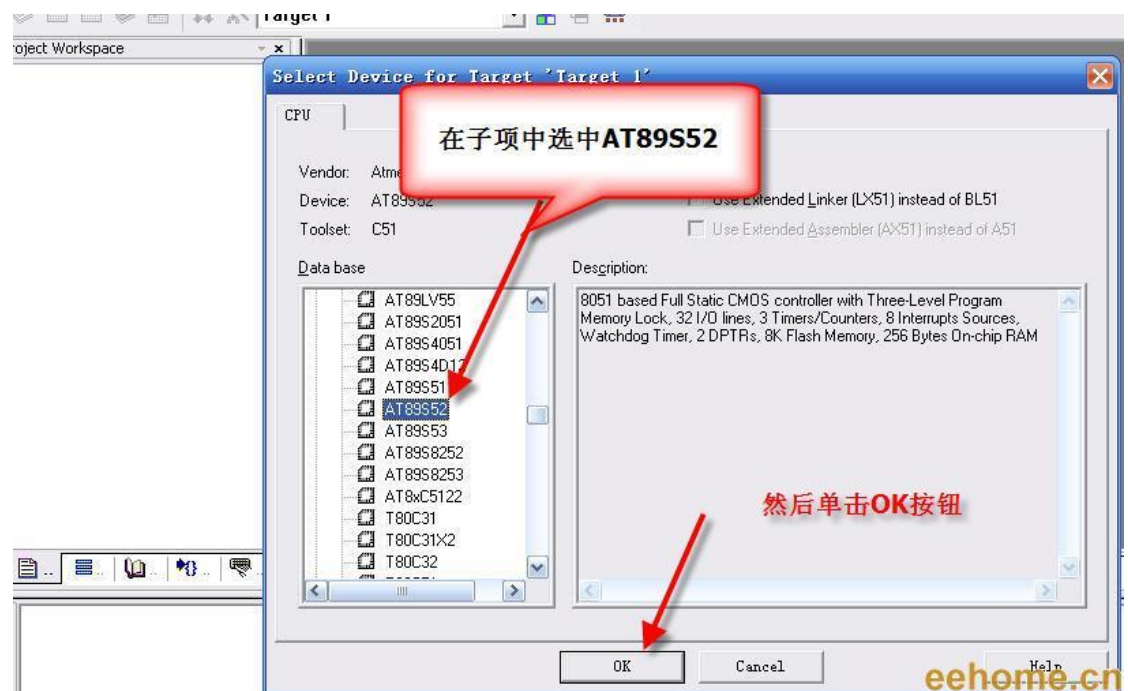
(原文件名:1.jpg)



(原文件名:2.jpg)



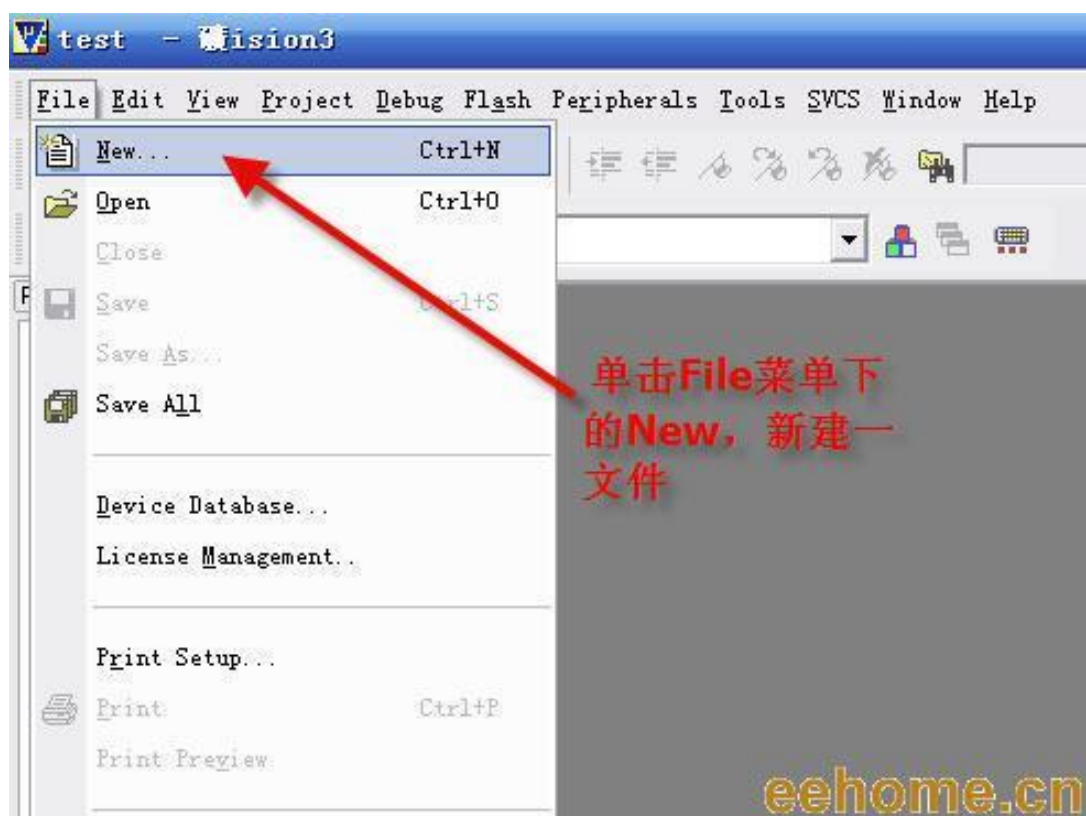
(原文件名:3.jpg)



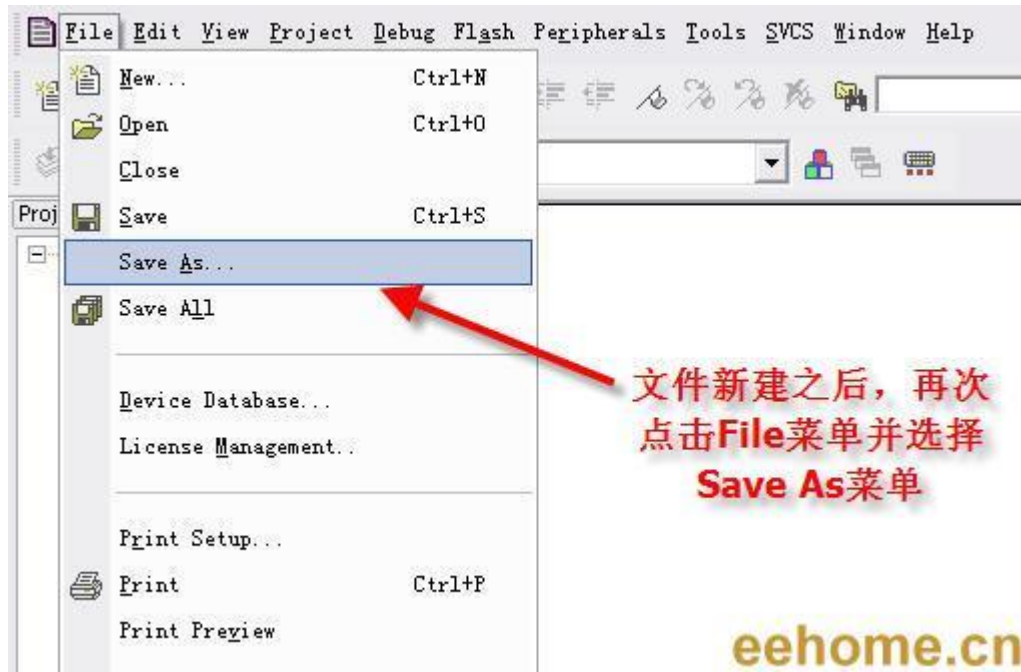
(原文件名:4.jpg)



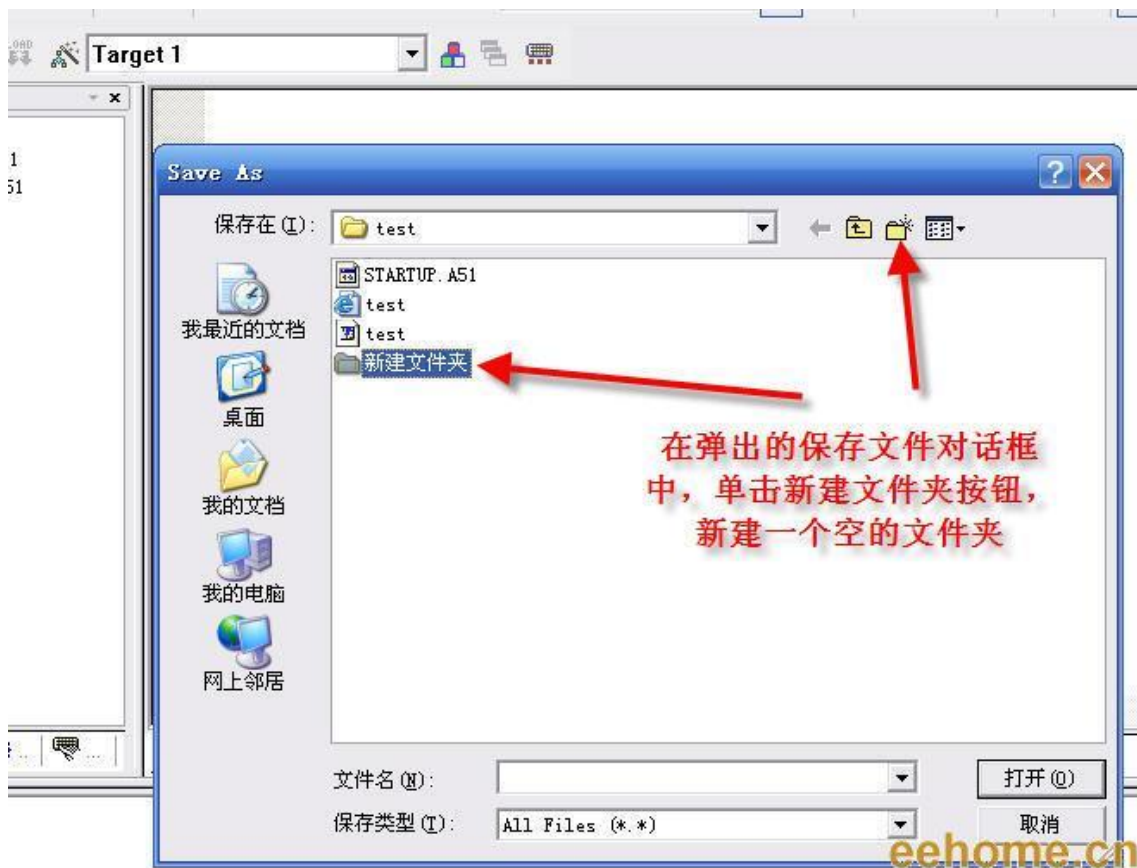
(原文件名:5.jpg)



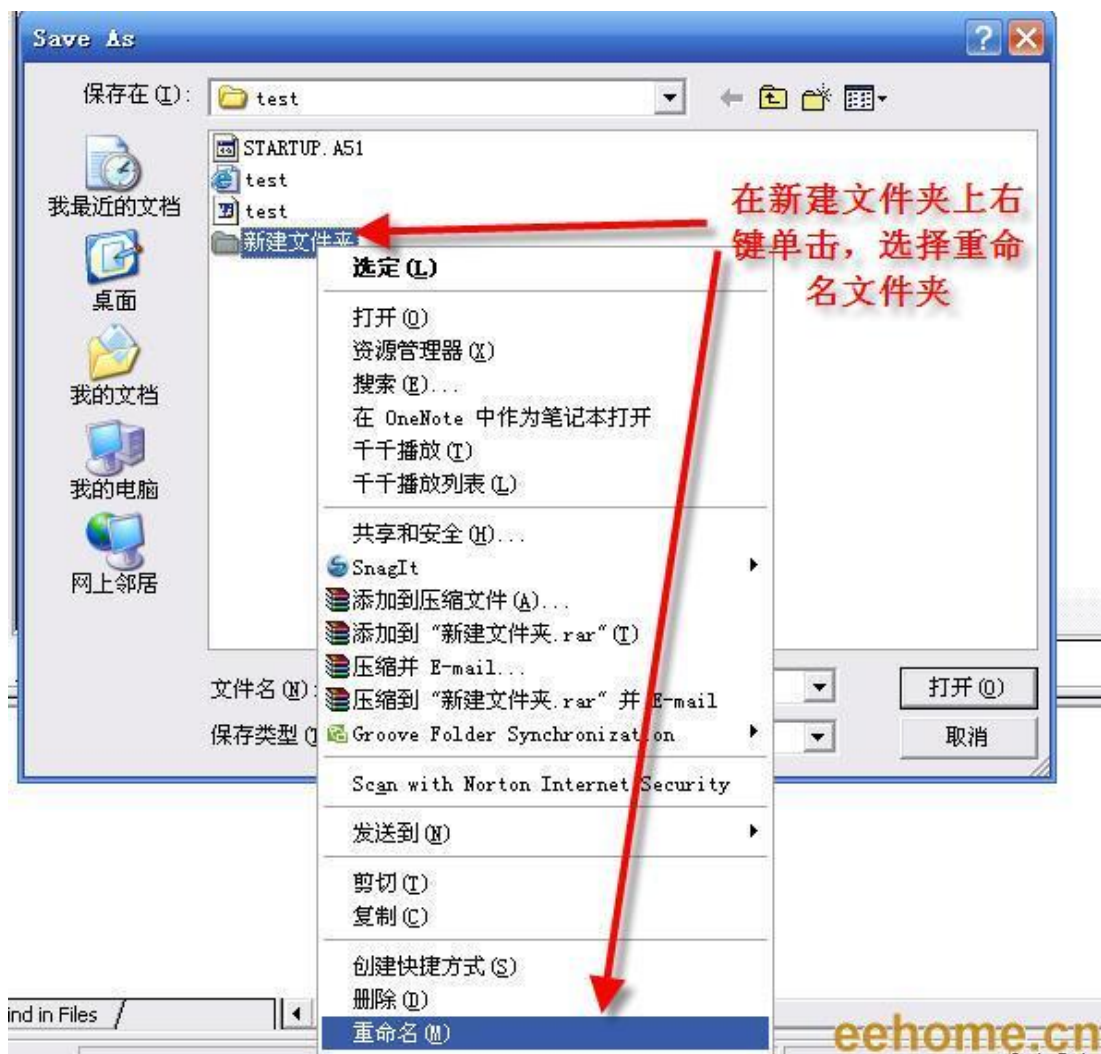
(原文件名:6.jpg)



(原文件名:7.jpg)



(原文件名:8.jpg)



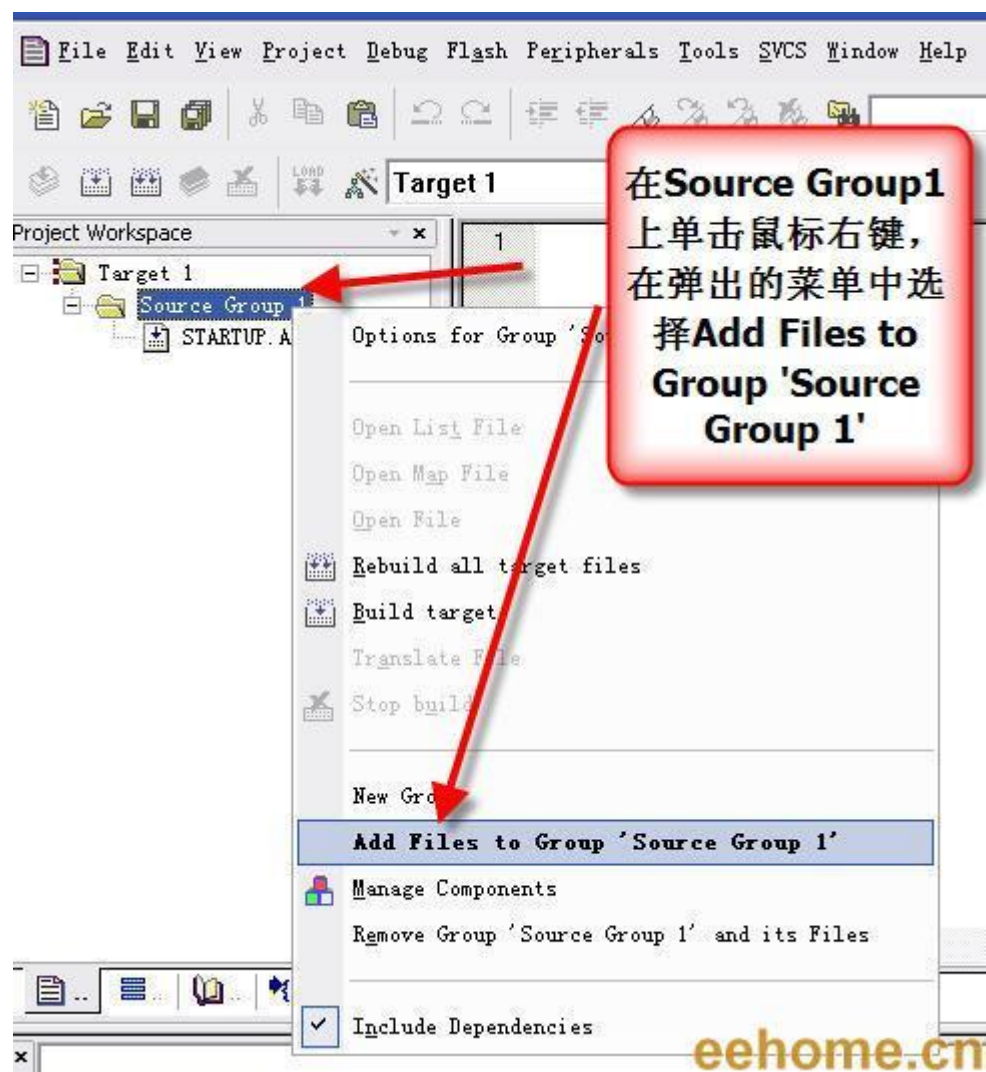
(原文件名:9.jpg)



(原文件名:10.jpg)



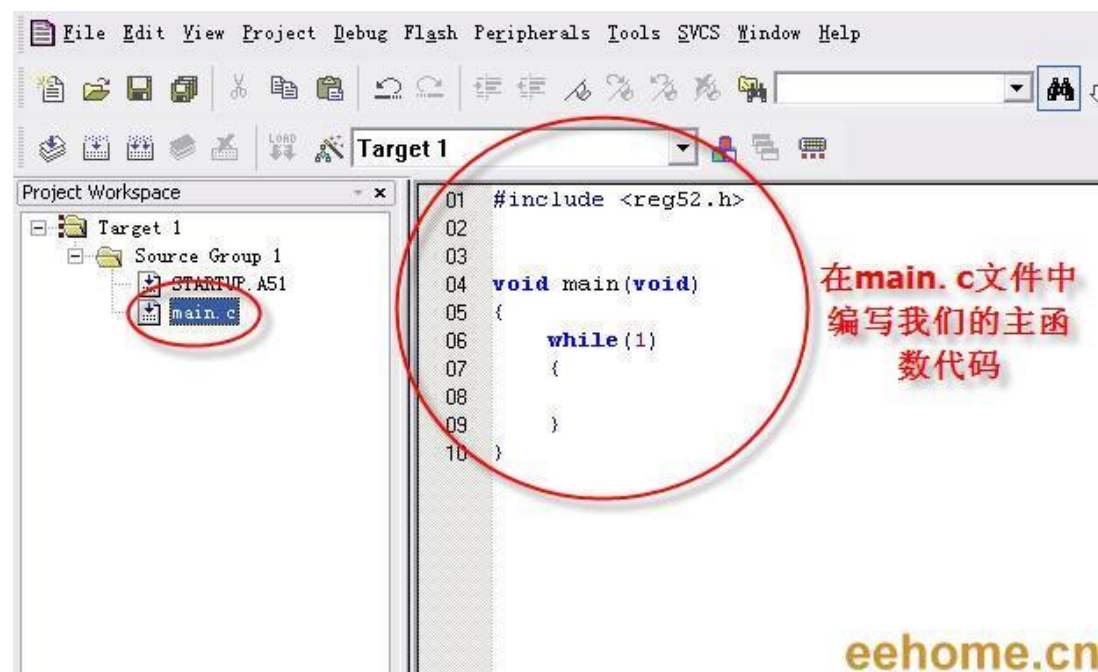
(原文件名:11.jpg)



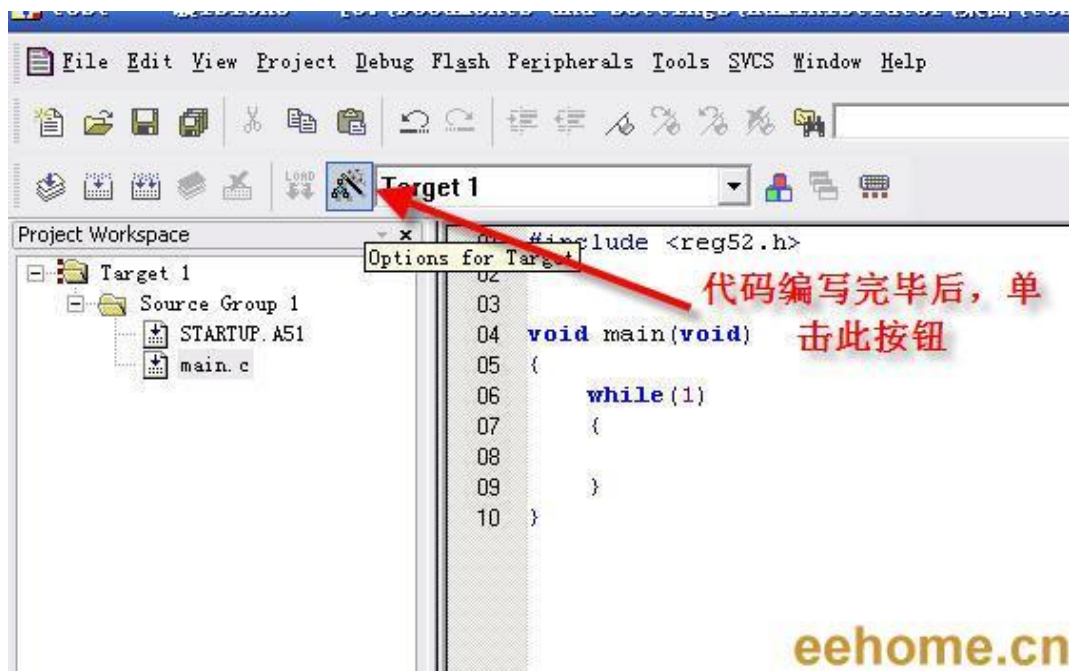
(文件名:12.jpg)



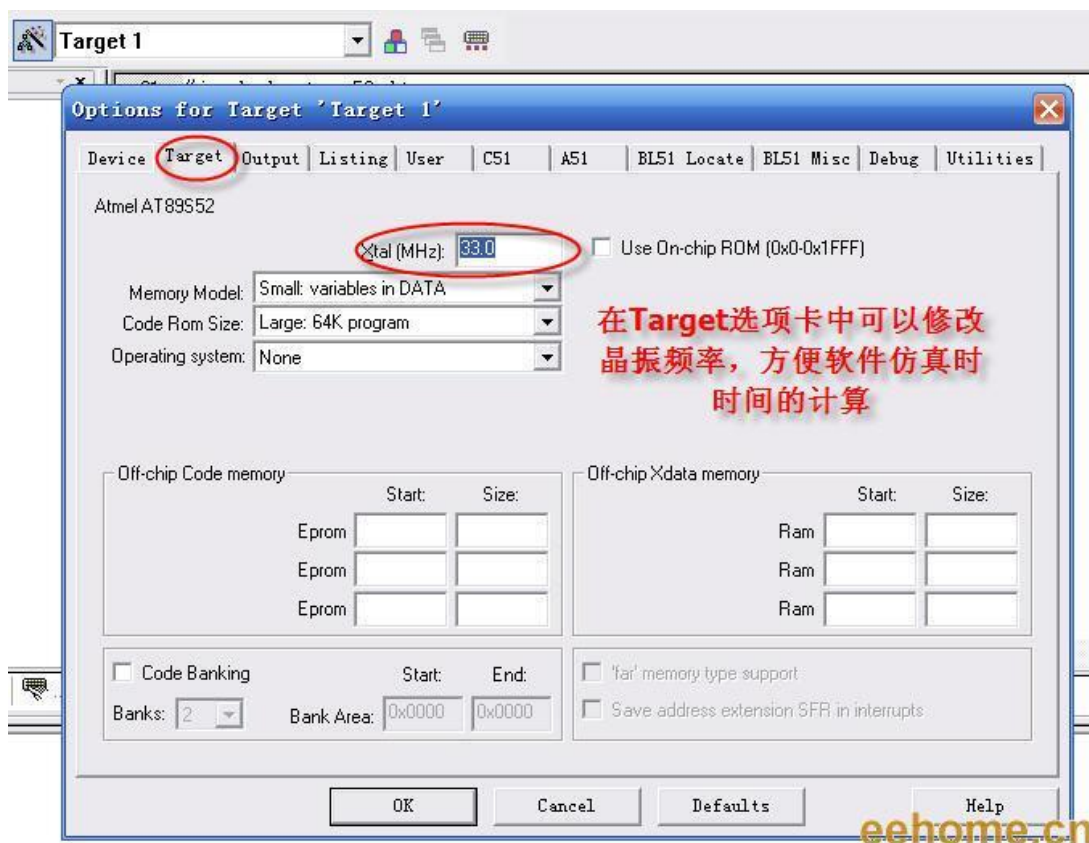
(原文件名:13.jpg)



(原文件名:14.jpg)



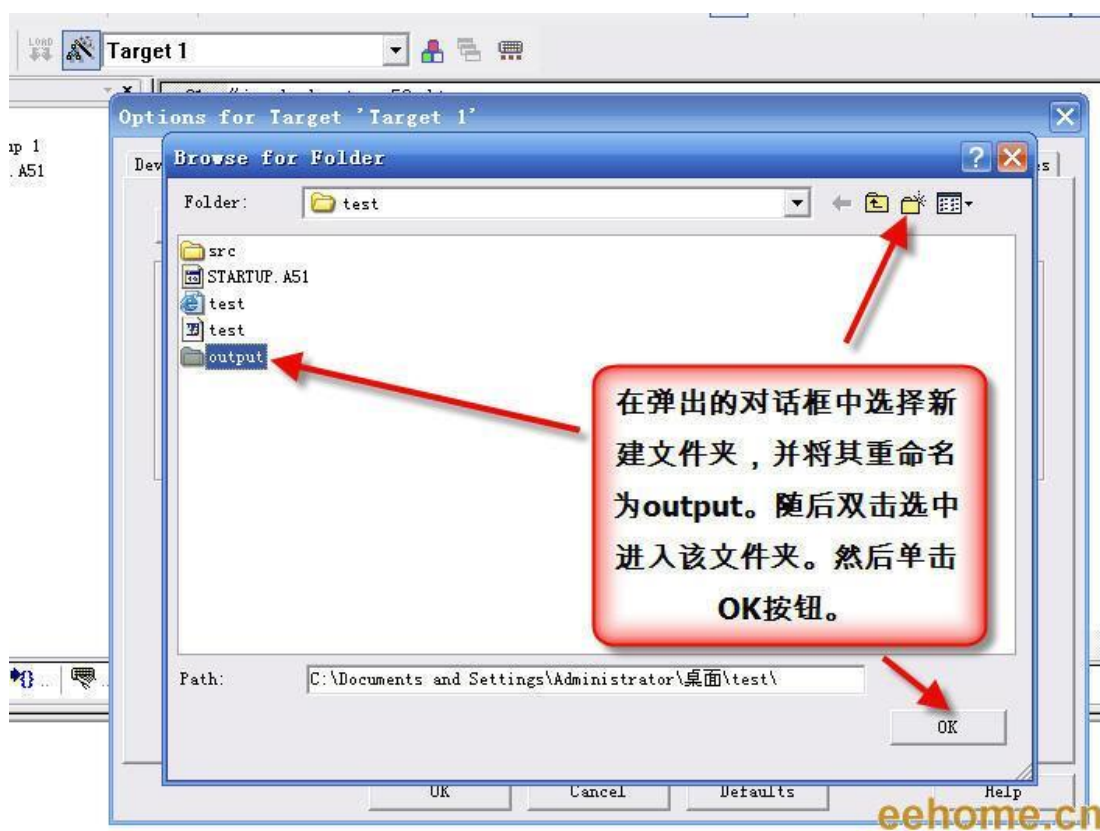
(原文件名:15.jpg)



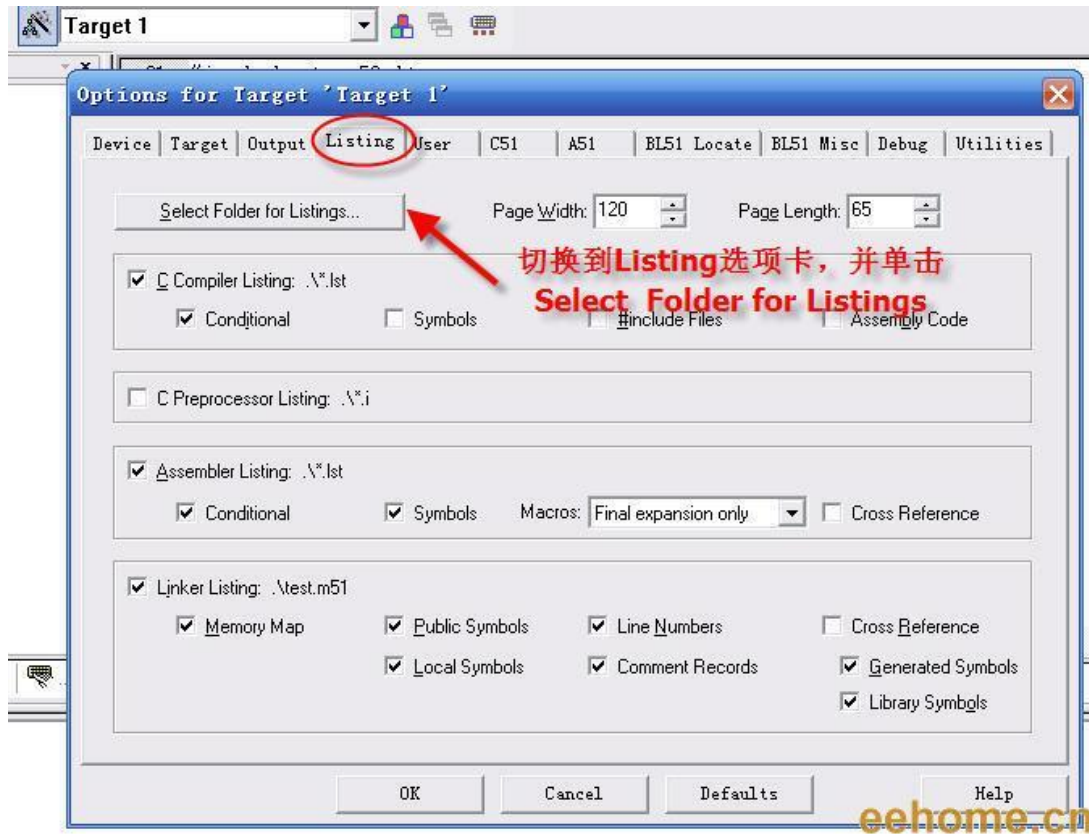
(原文件名:16.jpg)



(原文件名:17.jpg)



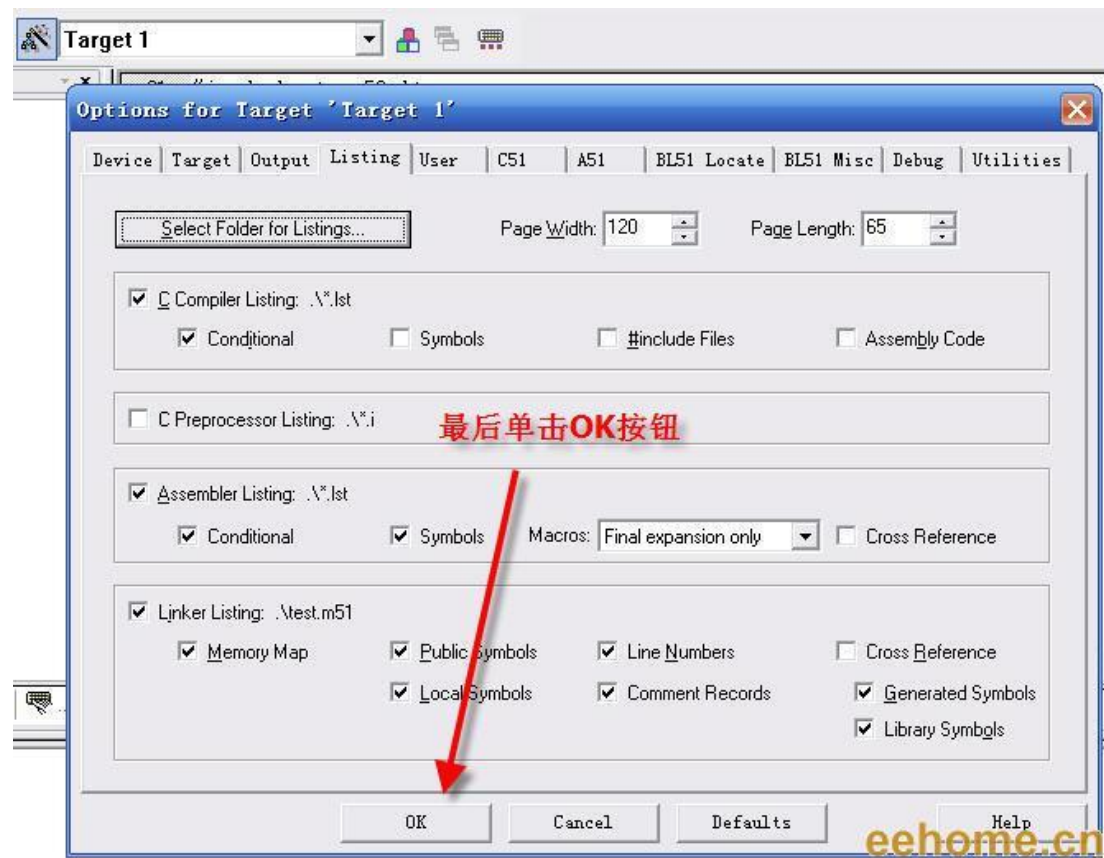
(原文件名:18.jpg)



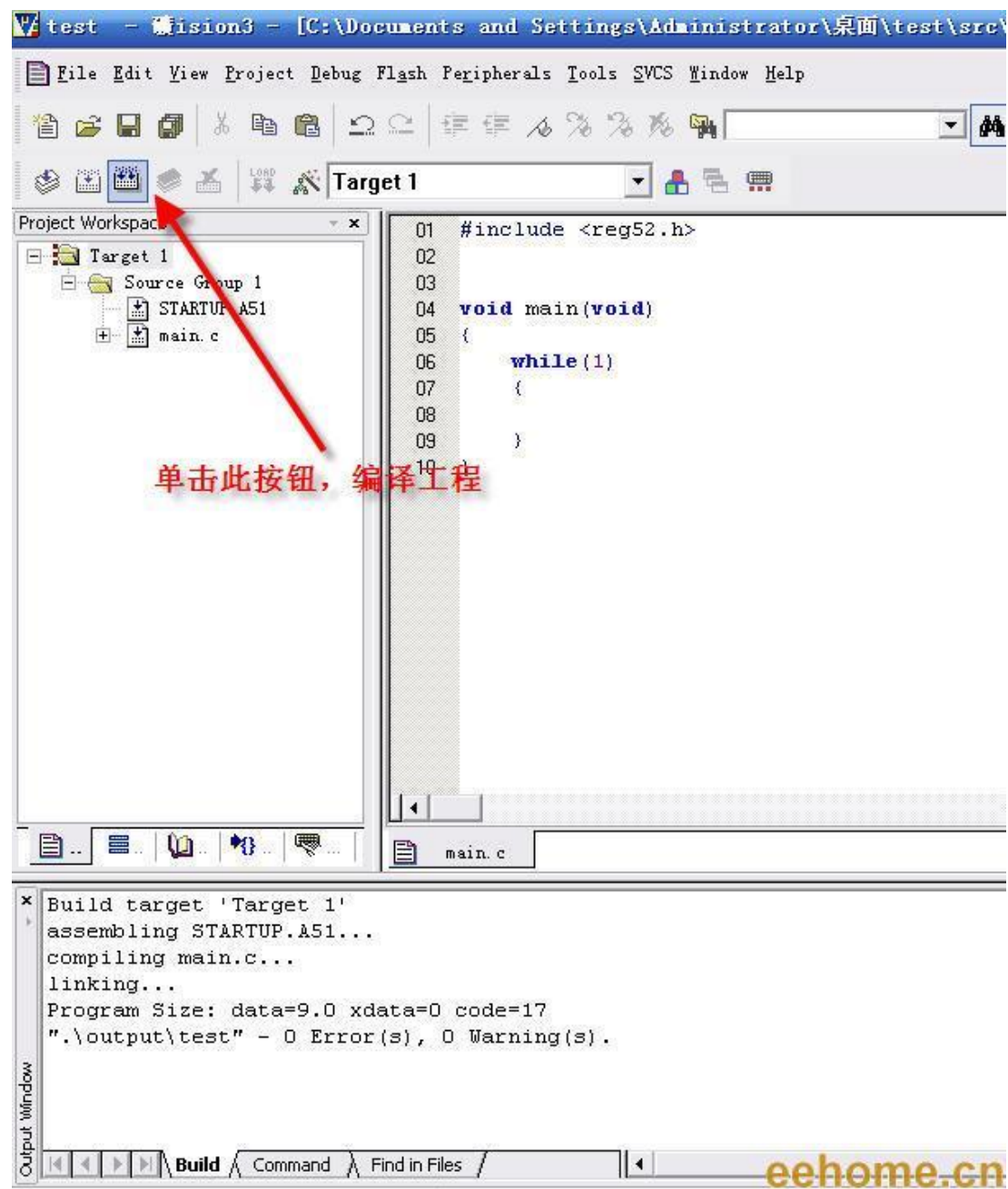
(原文件名:19.jpg)



(原文件名:20.jpg)



(原文件名:21.jpg)



(原文件名:22.jpg)

OK，到此一个简单的工程模板就建立起来了，以后我们再新建源文件和头文件的时候，就可以直接保存到 src 文件目录下面了。

下面我们开始编写各个模块文件。

首先编写 Timer.c 这个文件主要内容就是定时器初始化，以及定时器中断服务函数。其内容如下。

```
#include <reg52.h>
```

```
bit g_bSystemTime1Ms = 0;           // 1MS 系统时标
```

```

void Timer0Init(void)
{
    TMOD &= 0xf0 ;
    TMOD |= 0x01 ;    //定时器 0 工作方式 1
    TH0 = 0xfc ;    //定时器初始值
    TL0 = 0x66 ;
    TR0 = 1 ;
    ET0 = 1 ;
}

void Time0Isr(void) interrupt 1
{
    TH0 = 0xfc ;    //定时器重新赋初值
    TL0 = 0x66 ;
    g_bSystemTime1Ms = 1 ;    //1MS 时标标志位置位
}

```

由于在 Led.c 文件中需要调用我们的 g_bSystemTime1Ms 变量。同时主函数需要调用 Timer0Init()初始化函数，所以应该对这个变量和函数在头文件里作外部声明。以方便其它函数调用。

Timer.h 内容如下。

```

#ifndef _TIMER_H_
#define _TIMER_H_

extern void Timer0Init(void) ;
extern bit g_bSystemTime1Ms ;

#endif

```

完成了定时器模块后，我们开始编写 LED 驱动模块。

Led.c 内容如下：

```

#include <reg52.h>
#include "MacroAndConst.h"
#include "Led.h"
#include "Timer.h"

static uint16 g_u16LedTimeCount = 0 ; //LED 计数器
static uint8 g_u8LedState = 0 ;    //LED 状态标志, 0 表示亮, 1 表示熄灭

#define LED_P0    //定义 LED 接口
#define LED_ON()    LED = 0x00 ; //所有 LED 亮
#define LED_OFF()    LED = 0xff ; //所有 LED 熄灭

```



```

void LedProcess(void)
{
    if(0 == g_u8LedState) //如果 LED 的状态为亮，则点亮 LED
    {
        LED_ON();
    }
    else                //否则熄灭 LED
    {
        LED_OFF();
    }
}

```

```

void LedStateChange(void)
{
    if(g_bSystemTime1Ms)        //系统 1MS 时标到
    {
        g_bSystemTime1Ms = 0;
        g_u16LedTimeCount++;    //LED 计数器加一
        if(g_u16LedTimeCount >= 500) //计数达到 500,即 500MS 到了,改变 LED 的状态。
        {
            g_u16LedTimeCount = 0;
            g_u8LedState = !g_u8LedState ;
        }
    }
}

```

这个模块对外的借口只有两个函数，因此在相应的 Led.h 中需要作相应的声明。

Led.h 内容：

```

#ifndef _LED_H_
#define _LED_H_

extern void LedProcess(void);
extern void LedStateChange(void);

#endif

```

这两个模块完成后，我们将其 C 文件添加到工程中。然后开始编写主函数里的代码。如下所示：

```

#include <reg52.h>
#include "MacroAndConst.h"

```

```

#include "Timer.h"
#include "Led.h"

sbit LED_SEG = P1^4; //数码管段选
sbit LED_DIG = P1^5; //数码管位选
sbit LED_CS11 = P1^6; //led 控制位

void main(void)
{
    LED_CS11 = 1; //74HC595 输出允许
    LED_SEG = 0; //数码管段选和位选禁止(因为它们和 LED 共用 P0 口)
    LED_DIG = 0;
    Timer0Init();
    EA = 1;
    while(1)
    {
        LedProcess();
        LedStateChange();
    }
}

```

整个工程截图如下：

至此，第三章到此结束。
一起来总结一下我们需要注意的地方吧

1. C 语言源文件(*.c)的作用是什么
2. C 语言头文件(*.h)的作用是什么
3. typedef 的作用
4. 工程模板如何组织
5. 如何创建一个多模块(多文件)的工程

“从单片机初学者迈向单片机工程师”之 KEY 主题讨论
按键程序编写的基础

从这一章开始，我们步入按键程序设计的殿堂。在基于单片机为核心构成的应用系统中，用户输入是必不可少的一部分。输入可以分很多种情况，譬如有的系统支持 PS2 键盘的接口，有的系统输入是基于编码器，有的系统输入是基于串口或者 USB 或者其它输入通道等等。在各种输入途径中，更常见的是，基于单个按键或者由单个键盘按照一定排列构成的矩阵键盘(行列键盘)。我们这一篇章主要讨论的对象就是基于单个按键的程序设计，以及矩阵

键盘的程序编写。

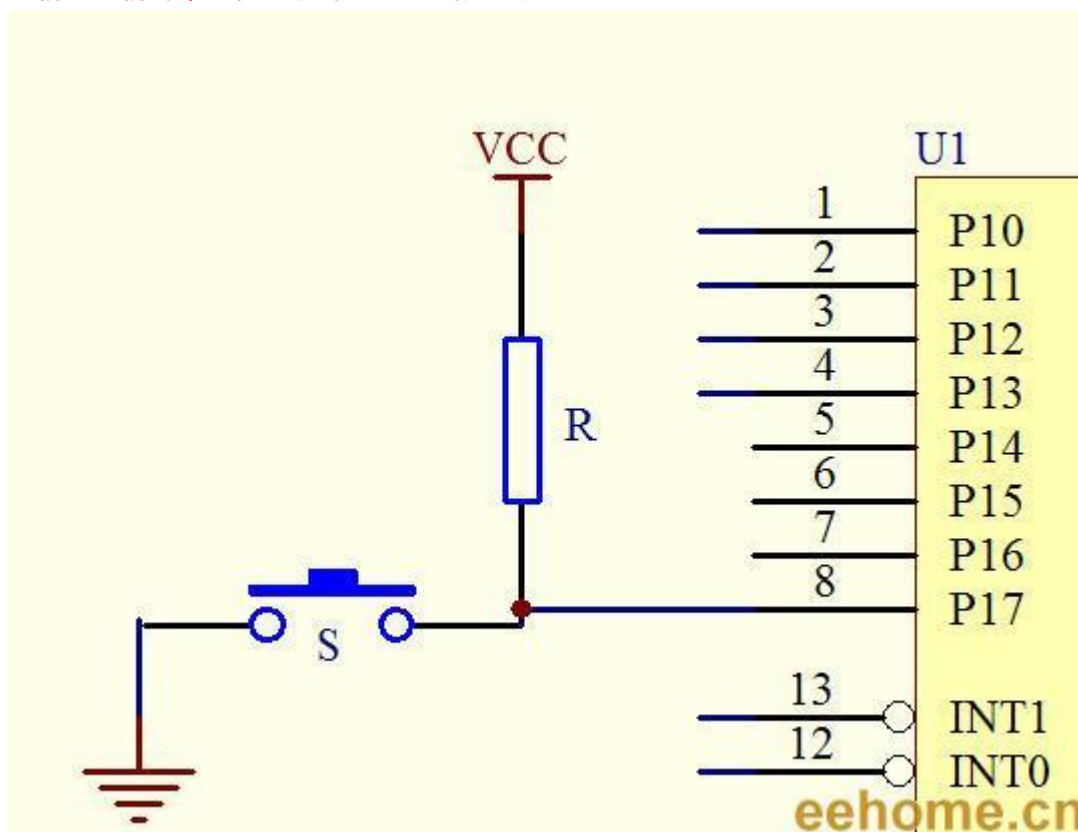
◎按键检测的原理

常见的独立按键的外观如下，相信大家并不陌生，各种常见的开发板学习板上随处可以看到他们的身影。



(文件名:1.jpg)

总共有四个引脚，一般情况下，处于同一边的两个引脚内部是连接在一起的，如何分辨两个引脚是否处在同一边呢？可以将按键翻转过来，处于同一边的两个引脚，有一条突起的线将他们连接一起，以标示它们俩是相连的。如果无法观察得到，用数字万用表的二极管挡位检测一下即可。搞清楚这点非常重要，对于我们画 PCB 的时候的封装很有益。它们和我们的单片机系统的 I/O 口连接一般如下：



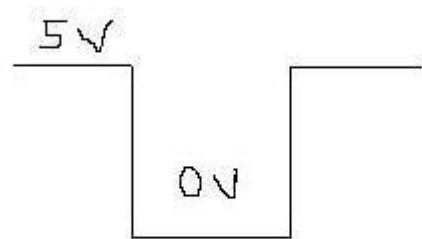
(文件名:2.jpg)

对于单片机 I/O 内部有上拉电阻的微控制器而言，还可以省掉外部的那个上拉电阻。简单分析一下按键检测的原理。当按键没有按下的时候，单片机 I/O 通过上拉电阻 R 接到 VCC，我们在程序中读取该 I/O 的电平的时候，其值为 1(高电平)；当按键 S 按下的时候，该 I/O 被短接到 GND，在程序中读取该 I/O 的电平的时候，其值为 0(低电平)。这样，按键的按下

与否，就和与该按键相连的 I/O 的电平的变化相对应起来。结论：我们在程序中通过检测到该 I/O 口电平的变化与否，即可以知道按键是否被按下，从而做出相应的响应。一切看起来很美好，是这样的吗？

◎现实并非理想

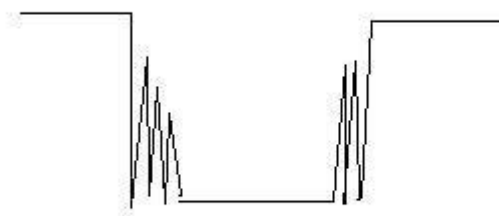
在我们通过上面的按键检测原理得出上述的结论的时候，其实忽略了一个重要的问题，那就是现实中按键按下时候的电平变化状态。我们的结论是基于理想的情况得出来的，就如同下面这幅按键按下时候对应电平变化的波形图一样：



eehome.cn

(原文件名:3.jpg)

而实际中，由于按键的弹片接触的时候，并不是一接触就紧紧的闭合，它还存在一定的抖动，尽管这个时间非常的短暂，但是对于我们执行时间以 us 为计算单位的微控制器来说，它太漫长了。因而，实际的波形图应该如下面这幅示意图一样。



eehome.cn

(原文件名:4.jpg)

这样便存在这样一个问题。假设我们的系统有这样功能需求：在检测到按键按下时，将某个 I/O 的状态取反。由于这种抖动的存在，使得我们的微控制器误以为是多次按键的按下，从而将某个 I/O 的状态不断取反，这并不是我们想要的效果，假如该 I/O 控制着系统中某个重要的执行的部件，那结果更不是我们所期待的。于是乎有人便提出了软件消除抖动的思想，道理很简单：抖动的时间长度是一定的，只要我们避开这段抖动时期，检测稳定的时候的电平不久可以了吗？听起来确实不错，而且实际应用起来效果也还可以。于是，各种各样的书籍中，在提到按键检测的时候，总也不忘说道软件消抖。就像下面的伪代码所描述的一样。(假设按键按下时，低电平有效)

```
if(0 == io_KeyEnter)      //如果有键按下了
{
    Delayms(20);          //先延时 20ms 避开抖动时期
    if(0 == io_KeyEnter)    //然后再检测，如果还是检测到有键按下
    {
        return KeyValue;    //是真的按下了，返回键值
    }
    else
    {
        return KEY_NULL     //是抖动，返回空的键值
    }
}
while(0 == io_KeyEnter);   //等待按键释放
}
```

乍看上去，确实挺不错，实际中呢？在实际的系统中，一般是不允许这么样做的。为什么呢？首先，这里的 Delayms(20)，让微控制器在这里白白等待了 20 ms 的时间，啥也没干，考虑我在《学会释放 CPU》一章中所提及的几点，这是不可取的。其次 while(0 == io_KeyEnter); 更是程序设计中的大忌(极少的特殊情况例外)。任何非极端情况下，都不要使用这样语句来堵塞微控制器的执行进程。原本是等待按键释放，结果 CPU 就一直死死的盯住该按键，其它事情都不管了，那其它事情不干了么？你同意别人可不会同意。所以合理的分配好微控制的处理时间，是编写按键程序的基础。

◎消除抖动有必要吗？

的确，软件上的消抖确实可以保证按键的有效检测。但是，这种消抖确实有必要吗？有人提出了这样的疑问。抖动是按键按下的过程中产生的，如果按键没有按下，抖动会产生吗？如果没有按键按下，抖动也会在 I/O 上出现，我会立刻把这个微控制器锤了，永远不用这样一款微控制器。所以抖动的出现即意味着按键已经按下，尽管这个电平还没有稳定。所以只要我们检测到按键按下，即可以返回键值，问题的关键是，在你执行完其它任务的时候，再次执行我们的按键任务的时候，抖动过程还没有结束，这样便有可能造成重复检测。所以，如何在返回键值后，避免重复检测，或者在按键一按下就执行功能函数，当功能函数的执行时

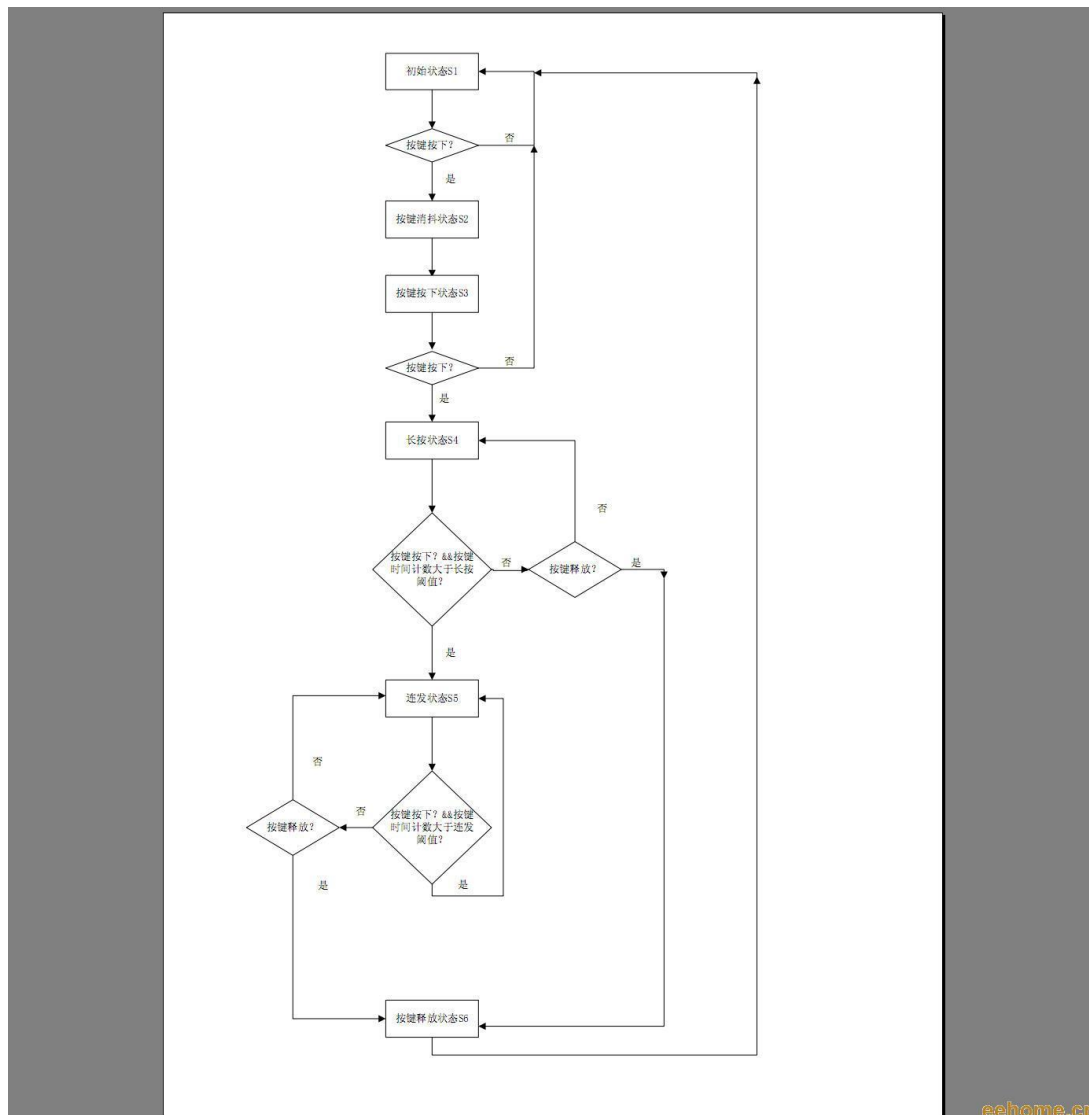
间小于抖动时间时候，如何避免再次执行功能函数，就成为我们要考虑的问题了。这是一个仁者见仁，智者见智的问题，就留给大家去思考吧。所以消除抖动的目的是：防止按键一次按下，多次响应。

“从单片机初学者迈向单片机工程师”之 KEY 主题讨论

基于状态转移的独立按键程序设计

本章所描述的按键程序要达到的目的：检测按键按下，短按，长按，释放。即通过按键的返回值我们可以获取到如下的信息：按键按下(短按)，按键长按，按键连_发，按键释放。不知道大家还记得小时候玩过的电子钟没有，就是外形类似于 CALL 机(CALL 机，好像是很古老的东西了)的那种，有一个小液晶屏，还有四个按键，功能是时钟，闹钟以及秒表。在调整时间的时候，短按+键每次调整值加一，长按的时候调整值连续增加。小的时候很好奇，这样的功能到底是如何实现的呢，今天就让我们来剖析它的原理吧。

状态在生活中随处可见。譬如早上的时候，闹钟把你叫醒了，这个时候，你便处于清醒的状态，马上你就穿衣起床洗漱吃早餐，这一系列事情就是你在这个状态做的事情。做完这些后你会去等车或者开车去上班，这个时候你就处在上班途中的状态……中午下班时间到了，你就处于中午下班的状态，诸如此类等等，在每一个状态我们都会做一些不同的事情，而总会有外界条件促使我们转换到另外一种状态，譬如闹钟叫醒我们了，下班时间到了等等。对于状态的定义出发点不同，考虑的方向不同，或者会有些许细节上面的差异，但是大的状态总是相同的。生活中的事物同样遵循同样的规律，譬如，用一个智能充电器给你的手机电池充电，刚开始，它是处于快速充电状态，随着电量的增加，电压的升高，当达到规定的电压时候，它会转换到恒压充电。总而言之，细心观察，你会发现生活中的总总都可以归结为一个个的状态，而状态的变换或者转移总是由某些条件引起同时伴随着一些动作的发生。我们的按键亦遵循同样的规律，下面让我们来简单的描绘一下它的状态流程转移图。



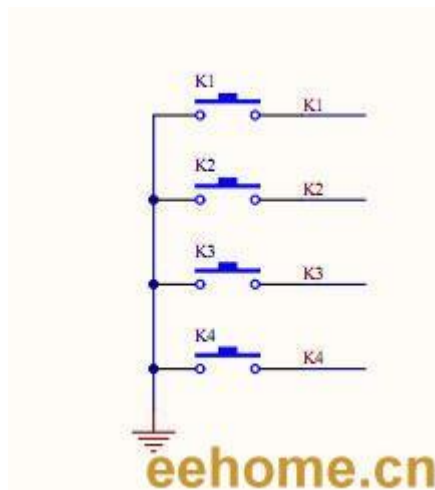
(原文件名:1.jpg)

下面对上面的流程图进行简要的分析。

首先按键程序进入初始状态 S1，在这个状态下，检测按键是否按下，如果有按下，则进入按键消抖状态 2，在下次执行按键程序时候，直接由按键消抖状态进入按键按下状态 3，在此状态下检测按键是否按下，如果没有按键按下，则返回初始状态 S1，如果有则可以返回键值，同时进入长按状态 S4，在长按状态下每次进入按键程序时候对按键时间计数，当计数值超过设定阈值时候，则表明长按事件发生，同时进入按键连_发状态 S5。如果按键键值为空键，则返回按键释放状态 S6，否则继续停留在本状态。在按键连_发状态下，如果按键键值为空键则返回按键释放状态 S6，如果按键时间计数超过连_发阈值，则返回连_发按键值，清零时间计数后继续停留在本状态。

看了这么多，也许你已经有一个模糊的概念了，下面让我们趁热打铁，一起来动手编写按键驱动程序吧。

下面是我使用的硬件的连接图。



(原文件名:2.jpg)

硬件连接很简单，四个独立按键分别接在 P3⁰-----P3³ 四个 I/O 上面。

因为 51 单片机 I/O 口内部结构的限制，在读取外部引脚状态的时候，需要向端口写 1.在 51 单片机复位后，不需要进行此操作也可以进行读取外部引脚的操作。因此，在按键的端口没有复用的情况下，可以省略此步骤。而对于其它一些真正双向 I/O 口的单片机来说，将引脚设置成输入状态，是必不可少的一个步骤。

下面的程序代码初始化引脚为输入。

```
void KeyInit(void)
{
    io_key_1 = 1 ;
    io_key_2 = 1 ;
    io_key_3 = 1 ;
    io_key_4 = 1 ;
}
```

根据按键硬件连接定义按键键值

```
#define KEY_VALUE_1      0x0e
#define KEY_VALUE_2      0x0d
#define KEY_VALUE_3      0x0b
#define KEY_VALUE_4      0x07
#define KEY_NULL         0x0f
```

下面我们来编写按键的硬件驱动程序。

根据第一章所描述的按键检测原理，我们可以很容易的得出如下的代码：

```
static uint8 KeyScan(void)
{
    if(io_key_1 == 0)return KEY_VALUE_1 ;
    if(io_key_2 == 0)return KEY_VALUE_2 ;
    if(io_key_3 == 0)return KEY_VALUE_3 ;
    if(io_key_4 == 0)return KEY_VALUE_4 ;
    return KEY_NULL ;
}
```

其中 io_key_1 等是我们按键端口的定义，如下所示：

```
sbit io_key_1 = P3^0 ;
sbit io_key_2 = P3^1 ;
sbit io_key_3 = P3^2 ;
sbit io_key_4 = P3^3 ;
```

KeyScan()作为底层按键的驱动程序，为上层按键扫描提供一个接口，这样我们编写的上层按键扫描函数可以几乎不用修改就可以拿到我们的其它程序中去使用，使得程序复用性大大提高。同时，通过有意识的将与底层硬件连接紧密的程序和与硬件无关的代码分开写，使得程序结构层次清晰，可移植性也更好。对于单片机类的程序而言，能够做到函数级别的代码重用已经足够了。

在编写我们的上层按键扫描函数之前，需要先完成一些宏定义。

//定义长按键的 TICK 数,以及连_发间隔的 TICK 数

```
#define KEY_LONG_PERIOD      100
#define KEY_CONTINUE_PERIOD  25
```

//定义按键返回值状态(按下,长按,连_发,释放)

```
#define KEY_DOWN              0x80
#define KEY_LONG              0x40
#define KEY_CONTINUE          0x20
#define KEY_UP                0x10
```

//定义按键状态

```
#define KEY_STATE_INIT        0
#define KEY_STATE_WOBBLE      1
#define KEY_STATE_PRESS       2
#define KEY_STATE_LONG        3
#define KEY_STATE_CONTINUE    4
#define KEY_STATE_RELEASE     5
```

接着我们开始编写完整的上层按键扫描函数，按键的短按，长按，连接，释放等等状态的判断均是在此函数中完成。对照状态流程转移图，然后再看下面的函数代码，可以更容易的去理解函数的执行流程。完整的函数代码如下：

```
void GetKey(uint8 *pKeyValue)
{
    static uint8 s_u8KeyState = KEY_STATE_INIT ;
    static uint8 s_u8KeyTimeCount = 0 ;
    static uint8 s_u8LastKey = KEY_NULL ; //保存按键释放时候的键值
    uint8 KeyTemp = KEY_NULL ;

    KeyTemp = KeyScan() ;      //获取键值

    switch(s_u8KeyState)
    {
```

```
case KEY_STATE_INIT :
{
    if(KEY_NULL != (KeyTemp))
    {
        s_u8KeyState = KEY_STATE_WOBBLE ;
    }
}
break ;

case KEY_STATE_WOBBLE :    //消抖
{
    s_u8KeyState = KEY_STATE_PRESS ;
}
break ;

case KEY_STATE_PRESS :
{
    if(KEY_NULL != (KeyTemp))
    {
        s_u8LastKey = KeyTemp ; //保存键值,以便在释放按键状态返回键值
        KeyTemp |= KEY_DOWN ; //按键按下
        s_u8KeyState = KEY_STATE_LONG ;
    }
    else
    {
        s_u8KeyState = KEY_STATE_INIT ;
    }
}
break ;

case KEY_STATE_LONG :
{
    if(KEY_NULL != (KeyTemp))
    {
        if(++s_u8KeyTimeCount > KEY_LONG_PERIOD)
        {
            s_u8KeyTimeCount = 0 ;
            KeyTemp |= KEY_LONG ; //长按键事件发生
            s_u8KeyState = KEY_STATE_CONTINUE ;
        }
    }
    else
    {
        s_u8KeyState = KEY_STATE_RELEASE ;
    }
}
```

```

        }
    }
    break ;

case KEY_STATE_CONTINUE :
    {
        if(KEY_NULL != (KeyTemp))
        {
            if(++s_u8KeyTimeCount > KEY_CONTINUE_PERIOD)
            {
                s_u8KeyTimeCount = 0 ;
                KeyTemp |= KEY_CONTINUE ;
            }
        }
        else
        {
            s_u8KeyState = KEY_STATE_RELEASE ;
        }
    }
    break ;

case KEY_STATE_RELEASE :
    {
        s_u8LastKey |= KEY_UP ;
        KeyTemp = s_u8LastKey ;
        s_u8KeyState = KEY_STATE_INIT ;
    }
    break ;

default : break ;
}
*pKeyValue = KeyTemp ; //返回键值
}

```

关于这个函数内部的细节我并不打算花过多笔墨去讲解。对照着按键状态流程转移图，然后去看程序代码，你会发现其实思路非常清晰。最能让人理解透彻的，莫非就是将整个程序自己看懂，然后想象为什么这个地方要这样写，抱着思考的态度去阅读程序，你会发现自己的程序水平会慢慢的提高。所以我更希望的是你能够认认真真的看完，然后思考。也许你会收获更多。

不管怎么样，这样的程序已经完成了本章开始时候要求的功能：按下，长按，连按，释放。事实上，如果掌握了这种基于状态转移的思想，你会发现要求实现其它按键功能，譬如，多键按下，功能键等等，亦相当简单，在下一章，我们就去实现它。

在主程序中我编写了这样的一段代码，来演示我实现的按键功能。

```

void main(void)
{

```

```

uint8 KeyValue = KEY_NULL;
uint8 temp = 0 ;
    LED_CS11 = 1 ; //流水灯输出允许
LED_SEG = 0 ;
LED_DIG = 0 ;
Timer0Init() ;
KeyInit() ;
EA = 1 ;
while(1)
{
    Timer0MainLoop() ;
    KeyMainLoop(&KeyValue) ;

    if(KeyValue == (KEY_VALUE_1 | KEY_DOWN)) P0 = ~1 ;
    if(KeyValue == (KEY_VALUE_1 | KEY_LONG)) P0 = ~2 ;
    if(KeyValue == (KEY_VALUE_1 | KEY_CONTINUE)) { P0 ^= 0xf0;}
    if(KeyValue == (KEY_VALUE_1 | KEY_UP)) P0 = 0xa5 ;
}
}

```

按住第一个键，可以清晰的看到 P0 口所接的 LED 的状态的变化。当按键按下时候，第一个 LED 灯亮，等待 2 S 后第二个 LED 亮，第一个熄灭，表示长按事件发生。再过 500 ms 第 5~8 个 LED 闪烁，表示连按事件发生。当释放按键时候，P0 口所接的 LED 的状态为：灭亮灭亮亮灭亮灭，这也正是 P0 = 0xa5 这条语句的功能。